Introduction à Git et GitHub

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Découverte de Git & GitHub **3**

A. Git 3

B. GitHub 5

C. Exercice : Quiz 5

III. Installation et configuration de Git **6**

A. Installer Git 6

B. Configurer GitHub 9

C. Exercice : Quiz 10

IV. L'essentiel **11**

V. Auto-évaluation **11**

A. Exercice 11

B. Test 11

Solutions des exercices **12**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 h

Environnement de travail : GitBash / Terminal

Prérequis : savoir comment fonctionne le terminal de commande

Contexte

Vous commencez à travailler sur un projet web et celui-ci devient de plus en plus conséquent. Le projet contenant des centaines de fichiers, vous ressentez le besoin de vérifier vos modifications en cours, d'annuler certaines d'entre elles, de travailler en équipe et de faire des sauvegardes de votre code. Mais comment faire ?

Nous allons pouvoir utiliser Git, qui est le logiciel de gestion de versions le plus connu. Mais alors, qu'est-ce que Git ? Comment ça marche ? Que peut-on faire avec ce logiciel ?

Nous allons répondre à toutes ces questions dans ce cours. La première partie de ce cours est une approche globale et théorique de ce que sont Git et GitHub. En deuxième partie, nous verrons comment installer Git, et comment utiliser GitHub.

II. Découverte de Git & GitHub

A. Git

Définition **Git et VCS**

Avant toute chose, il faut comprendre que Git est un outil que vous allez installer sur votre PC. L'objectif est d'avoir un outil pour mieux gérer le code source de votre projet.

Git est un système de gestion de versions (*version control system* ou VCS en anglais). Il est conçu pour suivre les modifications apportées au code source au fil du temps et pour faciliter la collaboration entre plusieurs développeurs sur un même projet. Il permet de sauvegarder, de suivre et de gérer les différentes versions de votre code, et de travailler à plusieurs sur un même projet sans écraser les modifications apportées par les uns et les autres (on appelle ça « *fusionner le code* »).

Exemple **Les problèmes résolus par les VCS**

- **La perte de modifications importantes :** sans un système de gestion de versions, il est facile de perdre les modifications importantes apportées au code. Par exemple, si un développeur fait une modification importante au code et que son ordinateur tombe en panne, ou pour une autre raison quelconque, il risque de perdre tout son travail.
- **La difficulté de suivre les modifications apportées au code :** il peut être difficile de suivre les modifications apportées au code au fil du temps. Les développeurs peuvent avoir des difficultés à savoir qui a modifié quoi, quand et pourquoi, ce qui peut rendre le code incompréhensible.
- **La difficulté de collaborer efficacement :** en effet, les développeurs peuvent modifier le même fichier en même temps, ce qui peut entraîner des conflits de code. Sans un système de gestion de versions pour gérer ces conflits, il peut être difficile pour les développeurs de travailler ensemble efficacement.
- **La difficulté de gérer les différentes versions du code :** sans un système de gestion de versions, il peut être difficile de gérer les différentes versions du code. Les développeurs peuvent avoir des difficultés à gérer celles-ci, ce qui peut rendre la maintenance et la correction des bogues plus difficile.
- **Facilité de déploiement sur des serveurs :** de nombreux outils ou sites de déploiement utilisent Git pour les déploiements, ce qui facilite de façon considérable les interactions entre le serveur et la machine du développeur.

Définition **Le vocabulaire de Git**

Vous avez peut-être déjà entendu parler des développeurs utilisant Git ; on pourrait croire qu'ils ont un langage bien à eux. C'est parce qu'il y a quelques concepts fondamentaux à comprendre avant de commencer l'utilisation de Git. Voici les principaux :

- Les **référentiels** Git (ou repo, pour *repository*) : un référentiel Git est un endroit où vous stockez votre code. Il peut être local (sur votre propre ordinateur) ou distant (sur un serveur distant).
- Les **commits** Git : un commit Git est un enregistrement des modifications apportées à votre code. Il contient un message de commit qui décrit les modifications apportées.
- Les **branches** Git : une branche Git est une version parallèle de votre code. C'est une copie de votre code complet, sur laquelle vous allez pouvoir travailler, sans modifier la version principale. Nous avons toujours au minimum une branche sur Git, la branche principale sur laquelle vous travaillez.

Méthode **Les outils pour travailler avec Git**

Git peut être utilisé en ligne de commande (Git CLI) ou avec une interface graphique. Généralement votre outil de développement embarque une interface graphique pour Git. C'est le cas de Visual Studio Code par exemple. Vous pouvez aussi installer des outils pour utiliser Git comme GitKraken, GitHub Desktop, Bitbucket, Sourcetree, etc.

Exemple **L'historique des modifications**

L'historique des modifications de Git permet de suivre les modifications apportées au code source au fil du temps. Lorsque vous travaillez sur un projet avec Git, chaque modification que vous apportez au code peut être enregistrée sous la forme d'un commit, qui, comme vu précédemment, enregistre les changements que vous avez effectués et les associe à un message de commit.

L'historique des modifications de Git permet de visualiser l'historique des commits effectués sur le projet, ainsi que les modifications apportées à chaque commit. Cela permet aux développeurs de suivre l'évolution du code au fil du temps, de voir qui a apporté des modifications et quand, et de revenir en arrière si nécessaire.

Voici une vue de l'historique des commits d'un projet :

```
commit 5f5d5c5c86d64317a1023f85e8f52c504d3ecb57
Author: John Doe <johndoe@example.com>
Date:   Thu Jun 3 14:08:08 2021 -0500

    Update index.html

commit 68c1c2f714b4df1cf2e29a4a4d36198b68e0b20a
Author: John Doe <johndoe@example.com>
Date:   Wed Jun 2 09:43:53 2021 -0500

    Add CSS to index.html

commit fef8317b53da76f8c7dcb2d3b3c3a53a9a9c3c85
Author: John Doe <johndoe@example.com>
Date:   Tue Jun 1 13:27:19 2021 -0500

    Initial commit
```

Sans connaître les fondamentaux, cela peut ne pas paraître si clair. Une interface graphique affichera ces informations de façon plus visuelle, mais nous pouvons déduire ces informations de cet historique :

- Le premier commit, intitulé « *Initial commit* », a été effectué le 1er juin 2021. Ce commit enregistre la première version du code, probablement un fichier HTML ou une structure de base pour le projet.

Tous droits réservés à STUDI - Reproduction interdite

- Le deuxième commit, intitulé « *Add CSS to index.html* », a été effectué le 2 juin 2021. Ce commit enregistre les modifications apportées au fichier HTML pour ajouter du style en utilisant des feuilles de style en cascade (CSS).
- Le troisième commit, intitulé « *Update index.html* », a été effectué le 3 juin 2021. Ce commit enregistre les modifications apportées au fichier HTML pour ajouter du contenu ou pour corriger des erreurs.

B. GitHub

Définition **Git**

Nous avions dit précédemment que Git était un système de gestion de versions. En fait, c'est plus que ça. Git est un système de gestion de versions **décentralisé**. Ce qui signifie que Git peut travailler avec une copie distante de lui-même. Vous avez sûrement déjà utilisé un cloud, c'est-à-dire un serveur qui vous permet de stocker vos données à distance, et donc de les récupérer depuis n'importe quelle machine. Git peut fonctionner de cette façon, en étant branché à une ***remote***, un cloud avec lequel il pourra interagir.

Ainsi, chaque développeur a une copie locale complète du code, ainsi qu'un accès au référentiel distant si nécessaire. Ils pourront donc tous travailler de manière indépendante sur différentes parties du code, tester des fonctionnalités et fusionner les modifications en toute sécurité.

Définition **GitHub**

GitHub est un service d'hébergement de code source basé sur Git. Notre machine avec Git en local pourra donc se connecter à GitHub pour stocker le code en ligne. Tout comme différents services de cloud existent (iCloud, OneDrive, Google Drive), différents services d'hébergement de code source existent (GitHub, GitLab, Azure DevOps, etc.).

Nous allons le voir, GitHub n'est pas qu'un simple serveur de stockage, il est beaucoup plus complet. Étant donné que c'est un service d'hébergement **basé sur Git**, nous allons pouvoir profiter de la puissance de Git sur ce serveur, mais ce n'est pas tout ! En plus de ça, GitHub propose énormément d'outils supplémentaires (gestion de projet, hébergement de site, gestion de tickets, gestion de documentation, etc.).

Méthode **Comment travailler avec GitHub ?**

Pour utiliser GitHub, vous devez d'abord créer un compte sur le site. Vous pouvez ensuite créer un référentiel (ou repository) pour votre projet et l'héberger sur GitHub. Vous pouvez également collaborer avec d'autres développeurs en ajoutant des contributeurs à votre projet, en créant des branches pour travailler sur des fonctionnalités ou des correctifs spécifiques, et en fusionnant les modifications en toute sécurité.

Lors de votre développement, il faudra régulièrement récupérer les modifications de vos collaborateurs, donc les télécharger depuis GitHub sur votre PC. C'est Git qui le fera pour vous *via* une commande. Il faudra que vous aussi, vous envoyiez régulièrement vos modifications sur le serveur, pour que vos collaborateurs accèdent régulièrement à vos modifications.

C. Exercice : Quiz

[solution n°1 p.13]

Question 1

Git est un système de gestion de versions centralisé.

- ○ Vrai
- ○ Faux

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Git permet de suivre les modifications apportées au code source au fil du temps.

- ○ Vrai
- ○ Faux

Question 3

Quel est le concept clé de Git qui permet de travailler sur des fonctionnalités ou des correctifs spécifiques sans affecter le code principal ?

- ○ Les référentiels Git
- ○ Les commits Git
- ○ Les branches Git

Question 4

Quel est le concept clé de Git qui est une sauvegarde de nos modifications de code ?

- ○ Les référentiels Git
- ○ Les commits Git
- ○ Les balises Git

Question 5

Git ne peut être utilisé qu'en ligne de commande.

- ○ Vrai
- ○ Faux

III. Installation et configuration de Git

A. Installer Git

Méthode **Télécharger et installer Git**

Pour installer Git, vous devez d'abord télécharger l'installeur à partir du site web officiel de Git[1].

Téléchargez la version correspondant à votre système d'exploitation.

- Windows[2]
- Mac[3]
- Linux[4]

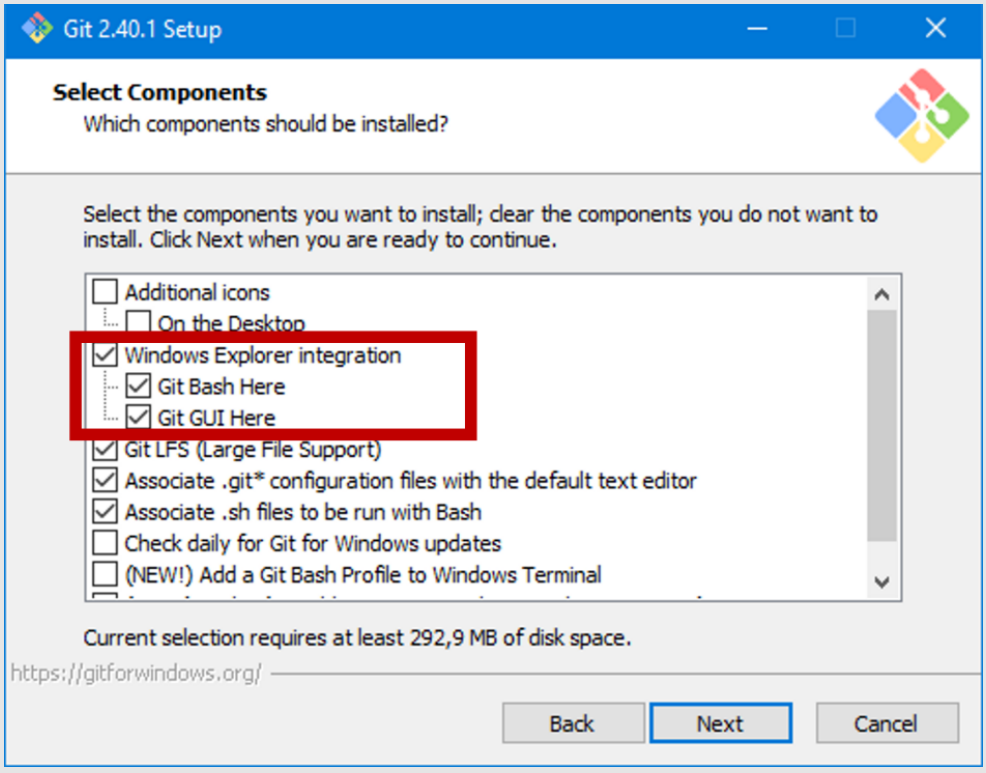
Une fois que vous avez téléchargé l'installeur, double-cliquez dessus pour lancer l'installation. Suivez les instructions à l'écran pour installer Git sur votre machine. Vous pouvez accepter les options par défaut ou personnaliser l'installation en fonction de vos besoins.

Voici les configurations que vous pouvez modifier ; pour les écrans non illustrés dans ce cours, vous pouvez laisser la configuration déjà proposée par l'installeur.

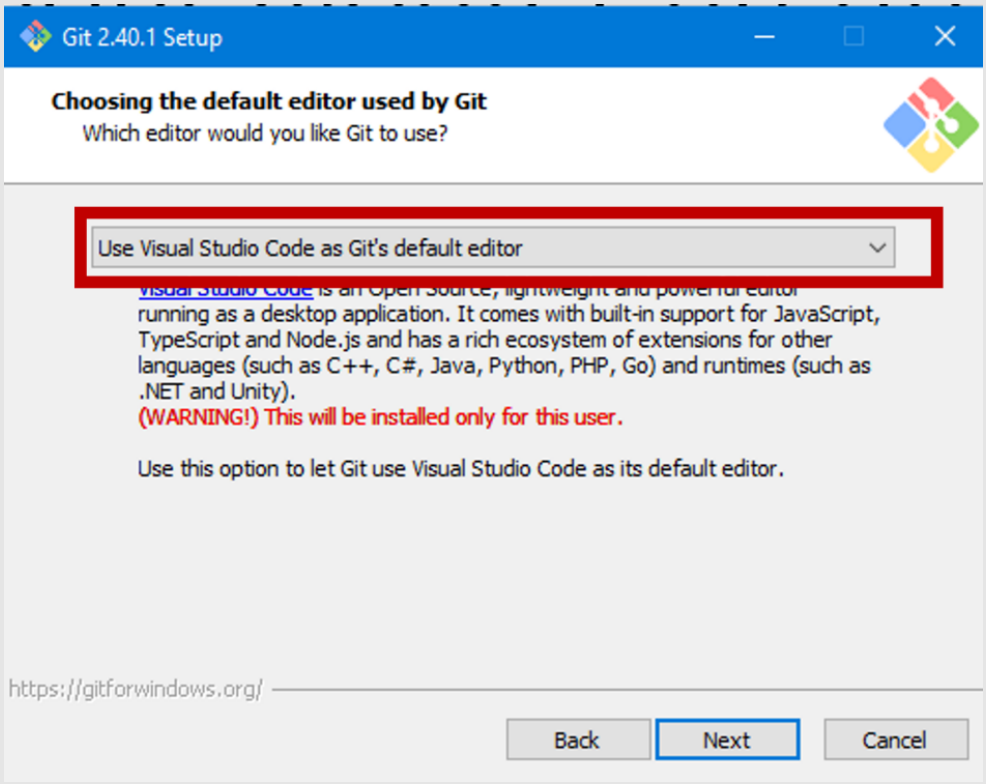
1 https://git-scm.com/downloads
2 https://git-scm.com/download/win
3 https://git-scm.com/download/mac
4 https://git-scm.com/download/linux

Tous droits réservés à STUDI - Reproduction interdite

Laissez les options « *Git Bash Here* » et « *Git GUI Here* ». Git Bash est une interface en ligne de commande pour Git. Il est conçu pour être utilisé sous Windows, afin de fournir une expérience de ligne de commande similaire à celle d'un système Unix. Ces options vous permettent d'accéder à ces outils directement depuis votre explorateur de fichier Windows.

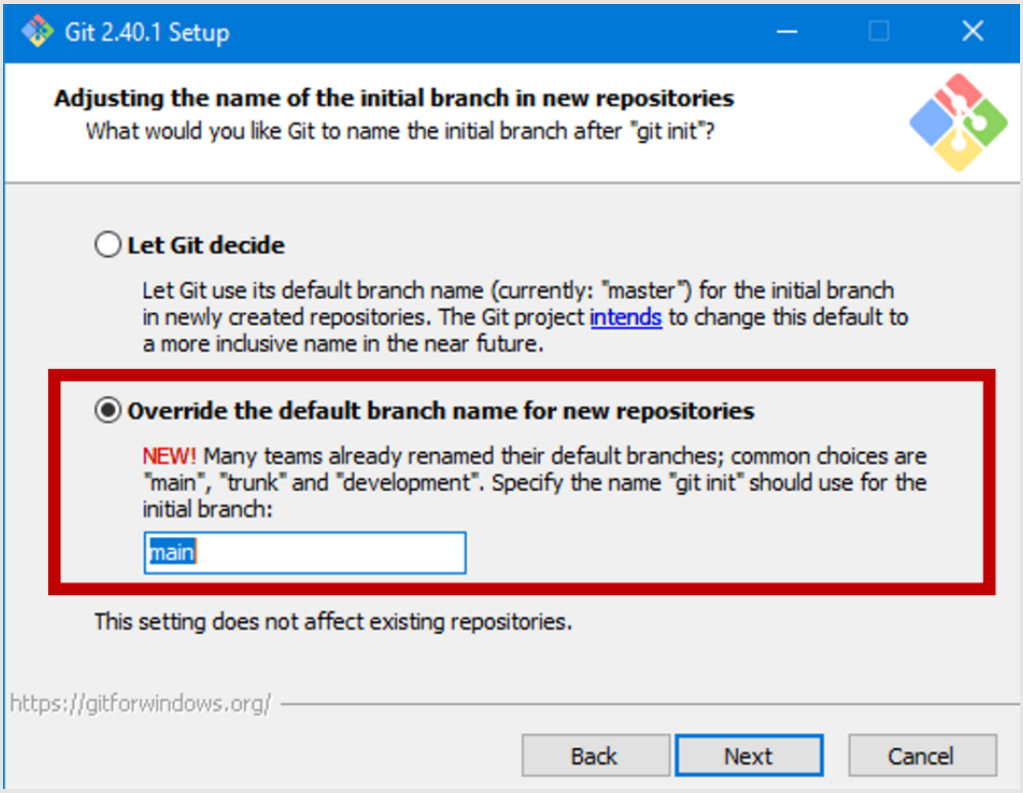


Choisissez l'éditeur de code par défaut. Git propose de choisir un éditeur de code par défaut. Si vous utilisez Visual Studio Code, vous pouvez choisir l'option « *Use Visual Studio Code as Git's default editor* ».

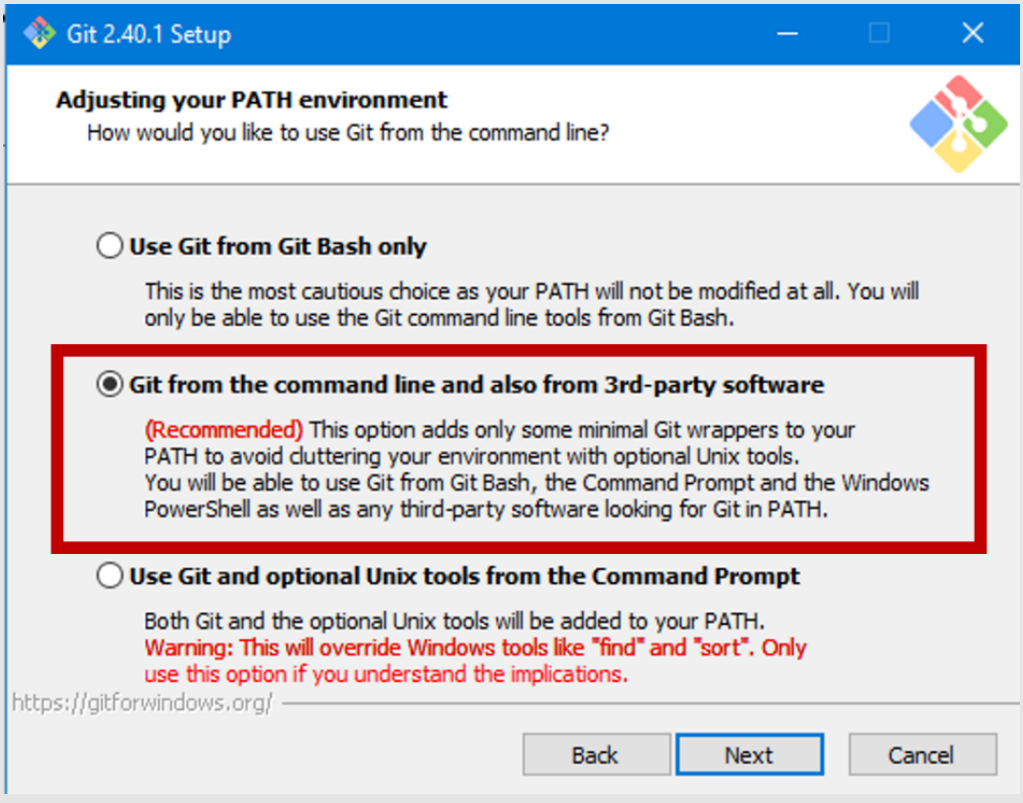


Tous droits réservés à STUDI - Reproduction interdite

Avant, on utilisait par défaut le nom de la branche principale « *master* », mais aujourd'hui, par défaut, on appelle plus souvent cette branche « *main* ». Vous pouvez donc cocher la case « *Override the default branch name for new repositories* » et spécifier le nom « *main* ».



Faites attention à bien avoir la case « *Git from the command line and also from 3rd-party software* » de cochée. Elle vous permettra de pouvoir utiliser Git *via* la commande `Git` depuis le terminal de commande.



Tous droits réservés à STUDI - Reproduction interdite

Après l'installation, vous pouvez vérifier si Git est correctement installé en ouvrant un terminal et en tapant la commande suivante :

```
git --version
```

Cette commande affichera la version de Git installée sur votre machine. Si vous avez votre version de Git, alors bravo, vous avez réussi !

B. Configurer GitHub

Méthode **Créer un compte GitHub**

Pour utiliser GitHub, vous devez d'abord créer un compte sur le site web officiel de GitHub[1]. Suivez les instructions à l'écran pour créer un compte. Le compte est gratuit. Pensez à vous donner un pseudo professionnel, car le compte GitHub sert parfois de carnet de démonstration de compétences.

Méthode **Configurer Git pour être lié à votre compte GitHub**

Il vous faut maintenant configurer votre Git pour lier votre Git local à votre compte GitHub. Lorsque vous voudrez interagir avec votre dépôt distant, vous n'aurez ainsi pas besoin de remettre vos informations de connexion à chaque fois. Pour cela, vous allez voir que c'est automatique.

Ce tutoriel de GitHub peut vous donner d'autres méthodes que celle que nous allons voir ci-dessus :

GiHub Docs[2]

Vous pouvez pour cela utiliser Git Credential Manager (GCM), qui s'installe automatiquement avec Git (au minimum, installez la version 2.29 ou supérieure, qui est la première version offrant le support OAuth pour GitHub).

Avec GCM, vous n'avez pas besoin de créer et de stocker manuellement un jeton d'accès personnel, car GCM gère l'authentification en votre nom, y compris la double authentification (2FA).

La prochaine fois que vous clonerez une URL HTTPS nécessitant une authentification, Git vous invitera à vous connecter *via* une fenêtre de navigateur. Vous pourriez d'abord être invité à autoriser une application OAuth. Si votre compte ou votre organisation exige une authentification à 2 facteurs, vous devrez également compléter l'authentification à 2 facteurs.

Une fois que vous vous êtes authentifié avec succès, vos identifiants sont stockés dans le gestionnaire d'identifiants de Windows et seront utilisés à chaque fois que vous clonerez une URL HTTPS. Git ne vous demandera pas de taper vos identifiants dans la ligne de commande à nouveau, à moins que vous ne modifiiez vos identifiants.

Exemple **Premiers pas avec Git**

Pour terminer ce cours, nous pouvons essayer d'utiliser Git, le but n'est pas encore de comprendre toutes les commandes, mais de voir comment on peut utiliser Git et GitHub. Nous allons créer un référentiel (repository), donc un dossier de travail sur GitHub, et nous allons le copier sur notre machine en local.

Pour créer un référentiel sur GitHub, connectez-vous à votre compte et cliquez sur le bouton « ***New repository*** ». Donnez un nom à votre référentiel, choisissez les options appropriées (public ou privé) et cliquez sur « ***Create repository*** ».

1 https://github.com/join
2 https://docs.github.com/en/get-started/getting-started-with-git/caching-your-github-credentials-in-git

Tous droits réservés à STUDI - Reproduction interdite

Il faut maintenant cloner ce référentiel sur votre machine. Vous aurez donc en finalité le référentiel sur votre machine, et le référentiel distant, les deux étant liés. Il faut savoir qu'un référentiel sur votre machine est en fait un dossier dans votre PC, que Git analysera. Vous pouvez donc créer un dossier à l'endroit de votre choix sur votre machine. Ouvrez ce dossier avec votre terminal de commande et entrez la commande suivante :

```
git clone "https://github.com/nom_utilisateur/nom_referentiel.gitHub"
```

Remplacez « *nom_utilisateur* » par votre nom d'utilisateur GitHub et « *nom_referentiel* » par le nom de votre référentiel. Vous pouvez aussi trouver cette URL directement sur GitHub.

Maintenant, vous pouvez ajouter des fichiers dans ce dossier, par exemple un fichier HTML. Une fois ce fichier ajouté, ouvrez un terminal et entrez les commandes suivantes :

```
git add .
git commit -m "Mon premier commit"
```

La commande `git add.` permet d'ajouter les fichiers modifiés ou nouveaux au prochain commit. La commande `git commit -m "Mon premier commit"` permet d'enregistrer ces modifications dans un commit avec un message associé.

Maintenant que vos modifications sont historiques, il faut les envoyer en ligne, donc sur GitHub.

Pour pousser les modifications sur GitHub, entrez la commande suivante :

```
git push origin main
```

Cette commande enverra les modifications sur le référentiel distant sur GitHub. Si vous allez sur votre référentiel sur GitHub, vous verrez votre fichier présent en ligne ! Vous avez fait votre premier référentiel, votre premier commit, et votre premier push.

C. Exercice : Quiz

[solution n°2 p.14]

Question 1

Git peut être installé sur Windows, Mac OS et Linux.

- Vrai
- Faux

Question 2

Quelle est la commande pour vérifier si Git est installé sur votre système ?

- `git version`
- `git install`
- `git check`

Question 3

Quelle est la commande pour configurer votre nom d'utilisateur dans Git ?

- `git config --global user.name "Votre nom"`
- `git config user.email "Votre nom"`
- `git config --global user.email "Votre nom"`

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Quelle est la commande pour cloner un dépôt GitHub sur votre ordinateur ?

- `git clone "URL du dépôt"`
- `git pull "URL du dépôt"`
- `git push "URL du dépôt"`

Question 5

Github est payant pour les repository privés, mais gratuit pour les publics.

- Vrai
- Faux

IV. L'essentiel

Ce cours sur Git et GitHub nous a permis de comprendre les bases de la gestion de version et de la collaboration en développement de logiciels.

Nous avons pu comprendre ce qu'est Git, ce qu'est GitHub, et voir comment ils interagissent entre eux. Nous comprenons maintenant que développer sans utiliser Git est un danger pour le bon déroulement du projet. Nous en avons besoin pour vérifier nos modifications, pour travailler à plusieurs, pour revenir en arrière sur certaines modifications, avoir des sauvegardes fiables, accessibles et performantes de notre code, etc.

Nous avons aussi appris à installer et configurer Git sur votre ordinateur, à créer un référentiel local, à ajouter des fichiers et à les envoyer sur GitHub. Maintenant que vous savez faire cela, vous pouvez approfondir le fonctionnement de Git pour qu'il n'ait plus de secrets pour vous, et gagner un temps précieux lors de vos futurs développements.

V. Auto-évaluation

A. Exercice

Vous êtes un développeur débutant qui vient de rejoindre une équipe de développement travaillant sur un projet. Vous vous rendez compte que cette équipe ne se met pas du tout à jour, et reste dans ses vieilles pratiques… Pire ! Elle ne connaît même pas Git, donc ils n'ont aucun outil de gestion de code. Vous en parlez à votre responsable, et il vous demande d'organiser une réunion qui répond aux questions ci-dessous. Pensez à écrire votre texte comme un PowerPoint pour une réunion.

Question 1

[solution n°3 p.15]

Qu'est-ce que Git et comment peut-il aider dans la gestion de version de code source ?

Indice :

N'hésitez pas à faire des recherches sur Internet pour avoir une vision plus exhaustive des capacités de Git.

Question 2

[solution n°4 p.15]

Pourquoi héberger mon code sur GitHub plutôt que sur un cloud classique ?

B. Test

Exercice 1 : Quiz

[solution n°5 p.16]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

À quoi sert Git ?

- Créer des applications web en 1 clic
- Gérer les versions de code pour les projets de développement
- Héberger votre site web

Question 2

Il faut installer GitHub sur sa machine pour l'utiliser.

- Vrai
- Faux

Question 3

Peut-on utiliser Git sans GitHub ?

- Non, Git nécessite GitHub pour fonctionner
- Oui, Git peut être utilisé indépendamment de GitHub
- Git et GitHub sont la même chose

Question 4

Git est peu utilisé dans le développement.

- Vrai
- Faux

Question 5

GitHub est une plateforme de développement collaborative pour les projets open source uniquement.

- Vrai
- Faux

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 5 Solution n°1

Question 1

Git est un système de gestion de versions centralisé.

- ○ Vrai
- ◉ Faux

Git est un système de gestion de versions décentralisé, ce qui signifie que chaque développeur a une copie locale complète du code, ainsi qu'un accès au référentiel distant si nécessaire.

Question 2

Git permet de suivre les modifications apportées au code source au fil du temps.

- ◉ Vrai
- ○ Faux

Git permet de sauvegarder, de suivre et de gérer les différentes versions du code source, ce qui facilite la gestion des modifications apportées au code au fil du temps. C'est même une des fonctionnalités les plus importantes de Git.

Question 3

Quel est le concept clé de Git qui permet de travailler sur des fonctionnalités ou des correctifs spécifiques sans affecter le code principal ?

- ○ Les référentiels Git
- ○ Les commits Git
- ◉ Les branches Git

Les branches Git permettent de travailler sur des fonctionnalités ou des correctifs spécifiques sans affecter le code principal. Cela permet aux développeurs de travailler de manière indépendante sur différentes parties du code, de tester des fonctionnalités et de fusionner les modifications en toute sécurité.

Question 4

Quel est le concept clé de Git qui est une sauvegarde de nos modifications de code ?

- ○ Les référentiels Git
- ◉ Les commits Git
- ○ Les balises Git

Un commit sur Git permet d'avoir un instantané du code à un moment précis dans le temps. Il enregistre les changements que vous avez effectués sur le code et les associe à un message de commit. Les commits permettent de suivre l'historique des modifications apportées au code, de visualiser qui a apporté des modifications et quand, et de revenir en arrière si nécessaire. En bref, un commit sur Git est une manière d'enregistrer les changements que vous avez apportés au code.

Question 5

Git ne peut être utilisé qu'en ligne de commande.

Tous droits réservés à STUDI - Reproduction interdite

- ○ Vrai
- ◉ Faux

Git peut être utilisé en ligne de commande ou avec une interface graphique. Vous pourrez trouver des outils d'interface graphique pour Git dans votre IDE, ou directement en les installant sur votre machine.

Exercice p. 10 **Solution n°2**

Question 1

Git peut être installé sur Windows, Mac OS et Linux.

- ◉ Vrai
- ○ Faux

Git est un système de gestion de versions distribué qui peut être installé sur les systèmes d'exploitation Windows, Mac OS et Linux.

Question 2

Quelle est la commande pour vérifier si Git est installé sur votre système ?

- ◉ `git version`
- ○ `git install`
- ○ `git check`

La commande `git version` vous permet de vérifier si Git est installé sur votre système et de voir la version de Git que vous avez installée.

Question 3

Quelle est la commande pour configurer votre nom d'utilisateur dans Git ?

- ◉ `git config --global user.name "Votre nom"`
- ○ `git config user.email "Votre nom"`
- ○ `git config --global user.email "Votre nom"`

La commande `git config --global user.name` vous permet de configurer votre nom d'utilisateur dans Git. Vous devez remplacer « *Votre nom* » par votre nom d'utilisateur. C'est ce nom d'utilisateur qui remontera sur vos commits en tant qu'auteur du commit.

Question 4

Quelle est la commande pour cloner un dépôt GitHub sur votre ordinateur ?

- ◉ `git clone "URL du dépôt"`
- ○ `git pull "URL du dépôt"`
- ○ `git push "URL du dépôt"`

La commande `git clone` vous permet de cloner un dépôt GitHub sur votre ordinateur. Vous devez remplacer « *URL du dépôt* » par l'URL du dépôt que vous souhaitez cloner.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Github est payant pour les repository privés, mais gratuit pour les publics.

- ○ Vrai
- ◉ Faux

Tous les repository sont gratuits sur GitHub. Il y a quelques années, ce n'était pas le cas, il fallait payer pour avoir des repository privés, mais depuis, GitHub a changé sa politique.

p. 11 Solution n°3

Git est un système de gestion de versions qui permet de garder une trace de l'historique des modifications apportées au code source. Il a énormément de fonctionnalité, et je vais vous en présenter aujourd'hui quelques-unes :

- Facilité de collaboration : Git permet de partager facilement les fichiers et les documents, de suivre les modifications apportées, de travailler ensemble sur une même version et de fusionner les changements.
- Historiques des modifications : Git permet de suivre les modifications apportées à un fichier ou à un ensemble de fichiers, ainsi que l'historique des versions. Si aujourd'hui on veut savoir qui a modifié un fichier le 3 mai par exemple, on ne peut pas, alors que Git le permet. On pourra même avoir un message lié à ces modifications, et on pourra voir ces modifications.
- Contrôle de version des documents : Git peut être utilisé pour suivre les modifications apportées à des documents, des présentations, des feuilles de calcul et d'autres fichiers. Cela permet de garder une trace des différentes versions des documents et d'éviter les problèmes de conflits lorsqu'il y a plusieurs personnes travaillant sur le même document.
- Sécurité : Git permet de sauvegarder les fichiers sur un serveur distant, assurant ainsi une protection contre les pertes de données. Il permet également de restaurer facilement une version précédente d'un fichier en cas de perte de données ou de modification accidentelle.
- Compatibilité multiplateforme : Git est compatible avec plusieurs systèmes d'exploitation tels que Windows, Mac OS et Linux, ce qui le rend accessible à un grand nombre d'utilisateurs. Donc pas de soucis pour la suite si on change nos outils de travail.

p. 11 Solution n°4

GitHub n'est pas un simple cloud, puisque c'est une plateforme de gestion de projets qui utilise Git comme système de gestion de versions. Il présente donc énormément de capacités qu'un cloud n'a pas. Voyons-en quelques-unes ensemble.

- Orienté développement : GitHub est une plateforme orientée développement et offre une variété d'outils pour les développeurs, tels que le suivi de versions, la gestion de code source, la collaboration en équipe et la gestion de problèmes. En revanche, un service de cloud classique est conçu pour stocker des fichiers de manière générale, sans offrir de fonctionnalités spécifiques pour les développeurs.
- Intégration : GitHub peut être intégré à une grande variété d'outils de développement, tels que des IDE (*Integrated Development Environment*) et des outils de gestion de projet. Cela peut faciliter le développement et la gestion de projets complexes. En revanche, les services de cloud classiques peuvent ne pas offrir cette intégration native.
- Prix : en fonction de votre utilisation, GitHub peut être gratuit.
- Sécurité : GitHub offre un niveau de sécurité supérieur à celui d'un service de cloud classique en termes de protection contre les pertes de données et d'attaques de sécurité. GitHub utilise des sauvegardes régulières et des protocoles de sécurité avancés pour protéger vos données.

Tous droits réservés à STUDI - Reproduction interdite

- Accessibilité : GitHub est accessible depuis n'importe quel ordinateur connecté à Internet, ce qui signifie que vous pouvez accéder à votre code et travailler sur votre projet depuis n'importe où dans le monde, de plus il est conçu pour stocker du code, il est habitué à stocker un très grand nombre de fichiers peu volumineux, là où un cloud stockera moins de fichiers, mais souvent plus volumineux. On aura ainsi de meilleures performances de téléchargement.

Exercice p. 11 **Solution n°5**

Question 1

À quoi sert Git ?

- ○ Créer des applications web en 1 clic
- ◉ Gérer les versions de code pour les projets de développement
- ○ Héberger votre site web

Git sert à gérer les versions de code, il permet d'avoir un historique de toutes les modifications, et de travailler à plusieurs sur un projet de développement.

Question 2

Il faut installer GitHub sur sa machine pour l'utiliser.

- ○ Vrai
- ◉ Faux

GitHub est une plateforme en ligne et n'a pas besoin d'être installée sur votre machine pour être utilisée. Cependant, vous aurez besoin de Git installé localement sur votre machine pour travailler avec des dépôts GitHub.

Question 3

Peut-on utiliser Git sans GitHub ?

- ○ Non, Git nécessite GitHub pour fonctionner
- ◉ Oui, Git peut être utilisé indépendamment de GitHub
- ○ Git et GitHub sont la même chose

Git est un système de gestion de versions distribué qui peut être utilisé indépendamment de GitHub. Bien que GitHub soit un outil de gestion de versions populaire, il n'est pas nécessaire pour utiliser Git sur votre machine. Vous pouvez utiliser Git sans repository distant, ou alors avec un autre outil que GitHub (GitLab, Azure DevOps, etc.).

Question 4

Git est peu utilisé dans le développement.

- ○ Vrai
- ◉ Faux

Git est l'un des systèmes de gestion de versions les plus populaires et les plus largement utilisés dans le développement de logiciels. De nombreuses entreprises et organisations utilisent Git pour gérer leurs projets de développement, et il est devenu un outil essentiel pour les développeurs.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

GitHub est une plateforme de développement collaborative pour les projets open source uniquement.

- ○ Vrai
- ◉ Faux

Bien que GitHub soit souvent utilisé pour les projets open source, il est également utilisé pour les projets privés et propriétaires.

Tous droits réservés à STUDI - Reproduction interdite